Midterm Exam

CAS CS 320: Concepts of Programming Languages

Name: Ankush Das

BUID:

- You will have approximately 75 minutes to complete this exam. Make sure to read every question, some are easier than others.
- Do not remove any pages from the exam.
- Make very clear what your final solution for each problem is (e.g., by surrounding it in a box). We reserve the right to mark off points if we cannot tell what your final solution is.
- You must show your work on all problems unless otherwise specified. A solution without work will be considered incorrect (and will be investigated for potential academic misconduct).
- Unless stated otherwise, you should only need the rules provided **in the problem statement** for any derivation.
- We will not look at any work on the pages marked "*This page is intentionally left blank*". You should use these pages for scratch work.

Question	Points
The Tests Pass	30
Breaking a Type System	20
Twice, Twice, Twice	25
Two Kinds of Lists	25
Total:	100

This page is intentionally left blank

1 The Tests Pass [30 points]

Your classmate was asked to implement a `dedup` function that takes a list as input and removes all *adjacent* duplicates from a list, e.g.,

```
dedup [2; 1; 1; 1; 2; 1; 2; 3; 3; 3; 5; 3; 3; 4] = [2; 1; 2; 1; 2; 3; 5; 3; 4]
```

Your classmate submits the following function:

```
let rec dedup l =
  match l with
  | x :: y :: rest -> if x = y then x :: dedup rest else x :: dedup (y :: rest)
  | _ -> l

let _ = assert (dedup [1; 1; 2] = [1; 2])
let _ = assert (dedup [1; 2; 2] = [1; 2])
let _ = assert (dedup [1; 1; 2; 2] = [1; 2])
let _ = assert (dedup [1; 2; 1] = [1; 2; 1])
```

All of the above tests pass.

(a) (10 points) Find an input of shortest possible length on which `dedup` gives the wrong answer, and state what it returns. Also, provide a brief justification.

Solution: `dedup [1; 1; 1]` returns `[1; 1]` instead of `[1]`. The first two elements match `x :: y :: rest` with `x = y = 1` and `rest = [1]`, so the result is `1 :: dedup [1]`, which is `[1; 1]`. Every list of length at most 2 is handled correctly, so 3 is the shortest failing length.

(b) (10 points) Describe exactly which inputs `dedup` gets wrong. In particular, describe exactly the output of `dedup` on a list of k equal adjacent elements.

Solution: A run of length k produces $\lceil k/2 \rceil$ copies, so every input with a run of length 3 or more is handled incorrectly. When the function finds an equal pair it skips both elements, and the kept `x` is never compared against the element that follows.

(c) (10 points) Rewrite the `dedup` function so that it removes adjacent duplicates correctly on all inputs. Write out the complete corrected function in OCaml.

Solution:

```
let rec dedup l =
  match l with
  | x :: y :: rest ->
    if x = y then dedup (x :: rest) else x :: dedup (y :: rest)
  | _ -> l
```

The only change is that `x :: dedup rest` becomes `dedup (x :: rest)` when `x = y`, so the kept element is compared against the element that follows.

2 Breaking a Type System [20 points]

Recall the typing and semantics rules for booleans, integers, and functions. The side conditions have been shown in boxes.

Typing.

$$\frac{\boxed{n \text{ is an integer literal}}}{\Gamma \vdash n : \texttt{int}}\ \textsc{intLit} \qquad \frac{}{\Gamma \vdash \texttt{true} : \texttt{bool}}\ \textsc{trueLit} \qquad \frac{}{\Gamma \vdash \texttt{false} : \texttt{bool}}\ \textsc{falseLit}$$

$$\frac{\boxed{(x : \tau) \in \Gamma}}{\Gamma \vdash x : \tau}\ \textsc{var} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \texttt{fun}\ x\ \texttt{->}\ e : \tau_1 \to \tau_2}\ \textsc{fun} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}\ \textsc{app}$$

$$\frac{\Gamma \vdash e_1 : \texttt{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau}\ \textsc{if}$$

Semantics.

$$\frac{\boxed{n \text{ is an integer literal}}}{n \Downarrow n}\ \textsc{intLitE} \qquad \frac{}{\texttt{true} \Downarrow \texttt{true}}\ \textsc{trueLitE} \qquad \frac{}{\texttt{false} \Downarrow \texttt{false}}\ \textsc{falseLitE}$$

$$\frac{}{\texttt{fun}\ x\ \texttt{->}\ e \Downarrow \texttt{fun}\ x\ \texttt{->}\ e}\ \textsc{funE} \qquad \frac{e_1 \Downarrow \texttt{fun}\ x\ \texttt{->}\ e \qquad e_2 \Downarrow v_2 \qquad \boxed{e' = [v_2/x]e} \qquad e' \Downarrow v}{e_1\ e_2 \Downarrow v}\ \textsc{appE}$$

$$\frac{e_1 \Downarrow \texttt{true} \qquad e_2 \Downarrow v}{\texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \Downarrow v}\ \textsc{ifTrueE} \qquad \frac{e_1 \Downarrow \texttt{false} \qquad e_3 \Downarrow v}{\texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 \Downarrow v}\ \textsc{ifFalseE}$$

For the questions in this section, you may only use the rules presented above.

(a) (10 points) Let e be the expression `(fun b -> if b then 0 else 1) true`. Determine a type τ and value v such that $\varnothing \vdash e : \tau$ and $e \Downarrow v$ are derivable. Also provide both derivations.

Solution: The type is $\tau = \texttt{int}$ and the value is $v = \texttt{0}$. Let $\Gamma' = b : \texttt{bool}$.

Typing derivation.

$$\dfrac{\dfrac{\dfrac{}{\Gamma' \vdash b : \texttt{bool}}\,\textsc{var} \quad \dfrac{}{\Gamma' \vdash \texttt{0} : \texttt{int}}\,\textsc{intLit} \quad \dfrac{}{\Gamma' \vdash \texttt{1} : \texttt{int}}\,\textsc{intLit}}{b : \texttt{bool} \vdash \texttt{if b then 0 else 1} : \texttt{int}}\,\textsc{if}}{\cdot \vdash \texttt{fun b -> if b then 0 else 1} : \texttt{bool} \to \texttt{int}}\,\textsc{fun} \quad \dfrac{}{\cdot \vdash \texttt{true} : \texttt{bool}}\,\textsc{trueLit}}{\cdot \vdash \texttt{(fun b -> if b then 0 else 1) true} : \texttt{int}}\,\textsc{app}$$

Let $f = \texttt{fun b -> if b then 0 else 1}$.

Evaluation derivation.

$$\dfrac{\dfrac{}{f \Downarrow f}\,\textsc{funE} \quad \dfrac{}{\texttt{true} \Downarrow \texttt{true}}\,\textsc{trueLitE} \quad \dfrac{\dfrac{}{\texttt{true} \Downarrow \texttt{true}}\,\textsc{trueLitE} \quad \dfrac{}{\texttt{0} \Downarrow \texttt{0}}\,\textsc{intLitE}}{\texttt{if true then 0 else 1} \Downarrow \texttt{0}}\,\textsc{ifTrueE}}{\texttt{(fun b -> if b then 0 else 1) true} \Downarrow \texttt{0}}\,\textsc{appE}$$

(b) (10 points) Your friend (who obviously has not taken CS 320) thinks that the APP rule is too restrictive and proposes replacing it with

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1'}{\Gamma \vdash e_1\ e_2 : \tau_2}\ \text{APP}'$$

An expression e is said to get *stuck* during evaluation if there is no value v such that $e \Downarrow v$. Find an expression e_0 such that $\varnothing \vdash e_0 : \tau$ is derivable for some type τ using APP′ (with all other rules the same) but gets stuck during evaluation. Provide a typing derivation for e_0, and briefly justify why evaluation gets stuck.

Solution: Take the expression `(fun x -> if x then 0 else 1) 1`. Let $\Gamma' = x$: `bool`.

$$\dfrac{\dfrac{\dfrac{}{\Gamma' \vdash x : \texttt{bool}}\ \text{VAR} \quad \dfrac{}{\Gamma' \vdash \texttt{0} : \texttt{int}}\ \text{INTLIT} \quad \dfrac{}{\Gamma' \vdash \texttt{1} : \texttt{int}}\ \text{INTLIT}}{x : \texttt{bool} \vdash \texttt{if x then 0 else 1} : \texttt{int}}\ \text{IF}}{\cdot \vdash \texttt{fun x -> if x then 0 else 1} : \texttt{bool} \to \texttt{int}}\ \text{FUN} \quad \dfrac{}{\cdot \vdash \texttt{1} : \texttt{int}}\ \text{INTLIT}$$
$$\cdot \vdash \texttt{(fun x -> if x then 0 else 1) 1} : \texttt{int}\ \text{APP}'$$

The argument has type `int` while the function expects a `bool`; APP′ allows this mismatch because τ_1' is unconstrained.

During evaluation, the function evaluates to `fun x -> if x then 0 else 1` by FUNE and argument `1` $\Downarrow$ `1` by INTLITE, so APPE substitutes `1` for x, giving `if 1 then 0 else 1`. This is not a well-typed expression. Neither of the IF rule applies, so the expression is stuck during evaluation.

3 Twice, Twice, Twice [25 points]

Consider the following OCaml functions where `twice` applies `f` to `x` twice.

```
let twice f x = f (f x)
let add m n = m + n
```

(a) (10 points) What is the type of `twice`?

Solution: `('a -> 'a) -> 'a -> 'a`

(b) (15 points) Note that for any integers `k` and `n`, the following equalities hold.

```
twice (add k) n = add (2 * k) n
twice twice (add k) n = add (4 * k) n
```

Determine the value of `twice twice twice (add 1) 0`. Also, provide a brief justification. *Hint.* Remember that function application is left associative.

Solution: 16. Function application associates to the left, so the expression is `((twice twice) twice) (add 1) 0`. We evaluate it step by step, using the definition `twice f x = f (f x)` and the two equalities given in the question.
The easiest way is to think of `twice` as `fun f -> fun x -> f (f x)`.

```
twice twice twice (add 1) 0
 = ((fun f -> fun x -> f (f x)) twice) twice (add 1) 0
 = (fun x -> twice (twice x)) twice (add 1) 0
 = (twice (twice twice)) (add 1) 0
 = (twice ((fun f -> fun x -> f (f x)) twice)) (add 1) 0
 = (twice (fun x -> twice (twice x))) (add 1) 0
```

Let g = `fun x -> twice (twice x)`. The previous line is

```
 = twice g (add 1) 0
 = (fun f -> fun x -> f (f x)) g (add 1) 0
 = (fun x -> g (g x)) (add 1) 0
 = g (g (add 1)) 0
 = g ((fun x -> twice (twice x)) (add 1)) 0
 = g (twice (twice (add 1))) 0
 = g (twice (add 2)) 0
 = g (add 4) 0
 = (fun x -> twice (twice x)) (add 4) 0
 = twice (twice (add 4)) 0
 = twice (add 8) 0 = add 16 0 = 16
```

This page is intentionally left blank

4 Two Kinds of Lists [25 points]

Consider the following OCaml program, which uses a user-defined list type `('a, 'b) mylist` containing elements of both types `'a` and `'b`:

```
let rec transform f g (l : ('a, 'b) mylist) =
  match l with
  | Nil -> Nil
  | ACons (a, xs) -> BCons (f a, transform f g xs)
  | BCons (b, ys) -> ACons (g b, transform f g ys)
```

(a) (10 points) The type `mylist` has two type parameters and is defined so that the program above type-checks. Fill in the definition.

```
type ('a, 'b) mylist = (* fill in here... *)
```

Solution:

```
type ('a, 'b) mylist =
  | Nil
  | ACons of 'a * ('a, 'b) mylist
  | BCons of 'b * ('a, 'b) mylist
```

A `mylist` is either `Nil`, or an element of type `'a` or of type `'b` followed by the rest of the list. I am choosing `ACons` with `'a` and `BCons` with `'b`.

(b) (15 points) What is the type of `transform` if you typed this code in `utop`, using your definition of `mylist` from part (a)? Provide a detailed explanation of how you arrived at this type.

Solution: `('a -> 'c) -> ('b -> 'd) -> ('a, 'b) mylist -> ('d, 'c) mylist`

- In the `ACons` case, `a : 'a`. The result `BCons (f a, ...)` uses `f a : 'c`.
- In the `BCons` case, `b : 'b`. The result `ACons (g b, ...)` uses `g b : 'd`.

Hence, the resulting list should be `('d, 'c) mylist` because `ACons` elements become `BCons` elements and the other way round.

This page is intentionally left blank